Core AI Architect

User Guide

A practical guide for defining an application blueprint (SSOT JSON) so an AI coding agent can build your app with clear instructions.

Coresynth Studios | SSOT authoring for Windows app builds

Contents

1. What this tool is (and is not)
2. Typical workflow
3. Menu reference
4. Section-by-section reference
    - 4.1 Project
    - 4.2 Environment
    - 4.3 Architecture
    - 4.4 UI (UI Layout Builder)
    - 4.5 Domain
    - 4.6 Workflows (Workflow Builder)
    - 4.7 Constraints
    - 4.8 Data
    - 4.9 Build
    - 4.10 Deployment
    - 4.11 Testing
    - 4.12 Documentation
5. Recommended fill order
6. Completeness checklist
7. After export - using the SSOT

1. What this tool is (and is not)

Core AI Architect does not write your app's code. It helps you author a structured blueprint (the SSOT JSON). You - or an AI coding agent - then use that blueprint to generate the real solution.

This tool does	This tool does not
Collect project choices in guided sections	Compile or publish the target app
Validate required fields	Invent screens/entities if you left them empty
Export JSON SSOT and Markdown docs	Replace Visual Studio or your coding agent

> **TIP**
> Empty lists (screens, entities, workflows, modules, etc.) mean "intentionally blank." A careful agent should not invent those for you. Fill the sections that matter for the app you want built.

2. Typical workflow

1. File -> New - pick a form profile (for example, WPF Desktop App).
2. Walk the left section list (Project through Documentation).
3. Use Previous / Next or click a section name.
4. Tools -> Validate when you think you are done.
5. File -> Save (keeps your SSOT) and/or Export JSON / Export Markdown.
6. Hand the .ssot.json (plus SSOT_CONTRACT.md) to your coding agent.

Form profiles

Profiles prefill environment defaults and tune available choices. They do not create demo content.

Profile	Best for
WPF Desktop App	Windows desktop GUI (WPF)
WinPE Tool	WinPE / recovery-style tools
Web API	ASP.NET APIs
CLI Tool	Console commands
PowerShell Module	Cmdlets / modules
Deployment Automation	Pipelines / orchestration
Custom	Broadest option sets (blank canvas)

3. Menu reference

Menu	Action	Purpose
File	New	Start a new SSOT with a profile
File	Open	Open an existing .json SSOT
File	Save / Save As	Persist your working SSOT file

File	Export JSON	Handoff file for coding agents
File	Export Markdown	Human-readable overview of filled sections
Tools	Validate	Check required fields and warnings
Tools	Copy PowerShell Command	Helper for PowerShell 7+ module use
Help	About / Module Help	Schemas path and orientation

4. Section-by-section reference

Each left-nav section becomes part of the exported SSOT. Together, these choices tell a coding agent exactly what kind of app to build. Each option below is shown as its own card so nothing overlaps.

4.1 Project

What this contributes to the built app: Identity and intent for the whole app (names, ownership, summary).

Project Id (Required)
Meaning: Machine-friendly key (prefer kebab-case), e.g. core-lottery-generator
Effect on built app: Filenames, package identity hints, stable references

Project Name (Required)
Meaning: Human title, e.g. Core Lottery Generator
Effect on built app: Window titles, README headings, branding

Description (Required)
Meaning: Plain-language purpose and scope
Effect on built app: Primary narrative for what to build

Owner (Optional)
Meaning: Person, team, or company
Effect on built app: Optional metadata in docs

Tags (Optional)
Meaning: Short labels (add one at a time)
Effect on built app: Categorization / search metadata

> **TIP**
> Put why and scope in Description. Put structured rules (entities, screens, workflows) in later sections - not only in the description paragraph.

4.2 Environment

What this contributes to the built app: Platform stack the generated solution must use.

Target OS (Required)
Meaning: Where the app runs (e.g. Windows 11)
Effect on built app: OS assumptions, APIs, packaging

Runtime (Required)

Meaning: Execution stack (e.g. .NET 8)

Effect on built app: SDK / target framework

UI Framework (Required)

Meaning: WPF, WinUI, Console, ASP.NET, etc.

Effect on built app: Project type and UI libraries

Language (Required)

Meaning: C#, PowerShell, or F#

Effect on built app: Language of generated code

TIP

Keep UI Framework consistent with UI -> Navigation Pattern (for example WPF + Window+Dialogs).

4.3 Architecture

What this contributes to the built app: How code is organized (pattern, layers, feature modules).

Pattern (Required)
Meaning: MVVM, MVC, Clean Architecture, etc.
Effect on built app: Folder layout, ViewModels vs controllers

Layers (Optional)
Meaning: Named layers as a list, e.g. UI, Domain, Infrastructure
Effect on built app: Keep those layers separate in code

Modules (Add Item) (Optional)
Meaning: Feature areas of the product
Effect on built app: Splits the solution into logical modules

Each module item

Name (Required for item)
Meaning: Short module name (e.g. Drawing)
Effect on built app: Module / folder / project naming

Description (Optional)
Meaning: What that area does
Effect on built app: Scope for that module's code

Dependencies (Optional)
Meaning: Other module names it needs
Effect on built app: References between modules

TIP
Modules are feature areas, not NuGet package names. Leave empty only if the app is tiny.

4.4 UI (UI Layout Builder)

What this contributes to the built app: Windows/screens, controls, navigation, and look - so the agent knows what to put on screen.

This section uses a dedicated UI Layout Builder (not only a simple form).

Theme (Required)
Meaning: Visual style (Default, Fluent, Custom, ...)
Effect on built app: Styling approach

Navigation Pattern (Required)
Meaning: How users move between surfaces
Effect on built app: Shell: dialogs, tabs, frame, routes

Screens (Strongly recommended)
Meaning: Each window/page/surface
Effect on built app: Creates those views

Controls (per screen) (Strongly recommended)
Meaning: Buttons, lists, inputs, etc.
Effect on built app: Concrete UI elements and actions

Each screen

Id (Required)
Meaning: Stable screen key (e.g. main)
Effect on built app: Code identifiers, navigation targets

Title (Required)
Meaning: User-visible title
Effect on built app: Window/page header

Route (Optional)
Meaning: Path or route hint (e.g. /)
Effect on built app: Navigation routing when applicable

Each control

Id (Required)

Meaning: Stable control key (e.g. generateButton)

Effect on built app: Bindings; workflow Trigger should match this

Type (Required)

Meaning: Button, TextBox, ComboBox, etc.

Effect on built app: Which UI widget to create

Label (Optional)

Meaning: Visible text

Effect on built app: Button/caption text

Action (Optional)

Meaning: Named behavior (e.g. GenerateNumbers)

Effect on built app: Maps click/command to a workflow

TIP

If Screens is empty, the agent has no UI contract - only your Description prose.

4.5 Domain

What this contributes to the built app: Data model - the "things" the app knows about.

Entities (Add Item) (Strongly recommended)
Meaning: Business objects (e.g. LotteryGame, DrawResult)
Effect on built app: Classes / records / tables

Each entity

Name (Required)
Meaning: Type name
Effect on built app: Class / table name

Description (Optional)
Meaning: What it represents
Effect on built app: Modeling guidance

Fields (Optional)
Meaning: Properties of the entity
Effect on built app: Typed properties with required flags

Each entity field

Name (Required)
Meaning: Property name
Effect on built app: Code property name

Type (Required)
Meaning: string, int, bool, decimal, DateTime, Guid
Effect on built app: Property / column type

Required (Optional)
Meaning: Must have a value
Effect on built app: Validation / non-nullability

TIP
Encode product rules here when possible (game name, ball count, min/max). Do not rely only on Project Description.

4.6 Workflows (Workflow Builder)

What this contributes to the built app: User/system processes - what happens when someone clicks Generate, Save, etc.

Uses a dedicated Workflow Builder.

Workflows (Add) (Strongly recommended)
Meaning: Named processes
Effect on built app: Commands / use-case methods

Each workflow

Id (Required)
Meaning: Stable key (e.g. generateNumbers)
Effect on built app: Code identifier

Name (Required)
Meaning: Display name
Effect on built app: Docs and UI labels

Trigger (Optional)
Meaning: What starts it (prefer a control Id from UI)
Effect on built app: Wires UI action to workflow

Steps (Optional)
Meaning: Ordered plain-language steps
Effect on built app: Implementation checklist for the agent

Error Strategy (Optional)
Meaning: How failures are handled
Effect on built app: MessageBox vs throw vs retry, etc.

Log Level (Optional)
Meaning: Severity for logging on errors
Effect on built app: Logging behavior

> **TIP**
> Fill UI controls first, then set Trigger to those control Ids so validation can warn on mismatches.

4.7 Constraints

What this contributes to the built app: Hard rules the agent must obey (libraries, logging, telemetry).

Allowed Libraries (Optional)

Meaning: Preferred packages (e.g. CommunityToolkit.Mvvm)

Effect on built app: Agent should prefer these when needed

Forbidden Libraries (Optional)

Meaning: Packages that must not be used

Effect on built app: Agent must not add them

Logging Required (Required)

Meaning: Whether logging must exist

Effect on built app: Adds logging if true

Telemetry Allowed (Required)

Meaning: Analytics/telemetry permission

Effect on built app: No telemetry if false

Third-Party Bootloaders Allowed (Required)

Meaning: Policy flag for bootloader-style third parties

Effect on built app: Agent must respect this policy

4.8 Data

What this contributes to the built app: How (and if) the app persists data.

Storage Type (Required)

Meaning: None, LocalJson, SQLite, SQLServer, ...

Effect on built app: Persistence technology

Connection String Mode (Required)

Meaning: Where connection/path config lives

Effect on built app: Config file vs env vs secrets vs none

Migrations Required (Required)

Meaning: Whether DB schema migrations are needed

Effect on built app: Migration projects/tooling if true

> **TIP**
> Local file history/settings: often LocalJson + connection mode None. Real database: SQLite/SQLServer + ConfigFile or UserSecrets.

4.9 Build

What this contributes to the built app: How the solution is compiled and for which platforms.

Output Type (Required)

Meaning: WinExe (GUI) vs Exe (console/host)

Effect on built app: Project output type

Configurations (Required)

Meaning: Usually Debug and Release

Effect on built app: Build configurations

Targets (Add Item) (Recommended)

Meaning: Named publish targets

Effect on built app: RIDs / publish profiles

Each target

Name (Required)

Meaning: Label (e.g. Desktop)

Effect on built app: Human name for the target

Runtime Identifier (Required)

Meaning: e.g. win-x64

Effect on built app: Publish RID

4.10 Deployment

What this contributes to the built app: How users receive the finished app.

Packaging (Required)

Meaning: None, MSIX, Zip, ...

Effect on built app: Packaging approach

Auto Update (Required)

Meaning: Whether auto-update is in scope

Effect on built app: Update mechanism (or none)

4.11 Testing

What this contributes to the built app: Expected automated test approach for the generated app.

Unit Test Framework (Required)

Meaning: xUnit, NUnit, MSTest, Pester, None

Effect on built app: Test project type

UI Test Framework (Required)

Meaning: None, FlaUI, WinAppDriver, Playwright

Effect on built app: UI automation stack

Coverage Target (Required)

Meaning: Number from 0 to 1 (e.g. 0.6 ~ 60%)

Effect on built app: Coverage goal for agents/CI

TIP

For desktop WPF/WinUI, prefer FlaUI or None - not Playwright (web-oriented) - unless you intentionally have a web UI.

4.12 Documentation

What this contributes to the built app: Which docs the agent (or Markdown export) should produce.

Generate README (Required)

Meaning: Want a README

Effect on built app: Includes overview-style README content

Generate Architecture Doc (Required)

Meaning: Want architecture documentation

Effect on built app: Architecture write-up when data exists

Sections (Optional)

Meaning: Overview, Architecture, Workflows, Entities, ...

Effect on built app: Which topics appear when data is filled

TIP

Empty product sections are omitted from Markdown export (no filler text).

5. Recommended fill order

For a complete agent handoff, fill sections in this order:

1. Project - Id, Name, Description
2. Environment - OS, Runtime, UI Framework, Language
3. Architecture - Pattern; add Layers/Modules if useful
4. UI - Theme, navigation, screens, controls
5. Domain - Entities and fields (rules as data)
6. Workflows - Processes tied to control Ids
7. Constraints - Libraries, logging, telemetry
8. Data - Storage choices
9. Build - Output type + at least one Target (win-x64)
10. Deployment - Packaging
11. Testing - Frameworks + coverage
12. Documentation - Flags/sections
13. Validate -> Save -> Export JSON

6. Completeness checklist

Before handing SSOT to an agent, confirm:

[ ] Project Id / Name / Description filled
[ ] Environment matches the kind of app you want
[ ] At least one screen with meaningful controls (for GUI apps)
[ ] Core entities defined (not only described in prose)
[ ] Key workflows with steps and triggers
[ ] Data/storage choice matches whether you need history/settings
[ ] Build target RID set (e.g. win-x64)
[ ] UI test framework makes sense for desktop vs web
[ ] Validate reports no unexpected errors

7. After export - using the SSOT

1. Keep your saved .ssot.json as the contract.
2. Point the coding agent at that file and docs/SSOT_CONTRACT.md.
3. Expect the agent to follow environment, UI, domain, workflows, and constraints strictly.
4. If something was left empty, either fill it in Core AI Architect and re-export, or explicitly tell the agent it may propose those parts.

Related files: README.md (build/run), docs/SSOT_CONTRACT.md (agent rules), docs/USER_GUIDE.md (Markdown source for this PDF).